

HOUSE OF HEARTS: ONLINE QUIZ

text

```
house-of-hearts-fullstack/
├─ backend/                               ← Spring Boot APIs
│   └─ pom.xml                           ← File 1
│   └─ src/main/java/...                 ← Files 2-15
│   └─ src/main/resources/...           ← Files 16-20
└─ frontend/                             ← React SPA
    └─ package.json                      ← File 21
    └─ public/index.html                 ← File 22
    └─ src/                             ← Files 23-25
```

BACKEND: Spring Boot APIs (20 Files)

FILE 1: pom.xml (backend/)

```
xml
Collapse Wrap Copy

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-parent</artifactId><version>2.1.6</version></parent>
  <groupId>com.example</groupId><artifactId>house-of-hearts-api</artifactId><version>0.0.1</version>
  <properties><java.version>21</java.version></properties>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId></dependency>
    <dependency><groupId>org.xerial</groupId><artifactId>sqlite-jdbc</artifactId><version>3.39.0</version></dependency>
    <dependency><groupId>org.hibernate.orm</groupId><artifactId>hibernate-community-dialects</artifactId><version>5.6.15</version></dependency>
  </dependencies>
  <build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId></plugin></plugins></build>
</project>
```

FILE 2: OnlineQuizApplication.java ([backend/src/main/java/com/example/onlinequiz/](#))

```
java ✖ Collapse ≡ Unwrap 📄 Copy  
  
package com.example.onlinequiz;  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
@SpringBootApplication  
public class OnlineQuizApplication { public static void main(String[] args) {  
SpringApplication.run(OnlineQuizApplication.class, args); } }
```

 FILE 3: Admin.java ([backend/src/main/java/com/example/onlinequiz/model/](#))

```
java ✖ Collapse ≡ Unwrap 📄 Copy  
  
package com.example.onlinequiz.model; import jakarta.persistence.*;  
@Entity @Table(name="admins") public class Admin {  
    @Id @GeneratedValue private Long id; private String username, password;  
    public Admin() {} public Admin(String username, String password) {  
this.username=username; this.password=password; }  
    // Getters/Setters  
    public Long getId() { return id; } public void setId(Long id) { this.id = id; }  
    public String getUsername() { return username; } public void setUsername(String  
username) { this.username = username; }  
    public String getPassword() { return password; } public void setPassword(String  
password) { this.password = password; }  
}
```

FILE 4: House.java ([backend/src/main/java/com/example/onlinequiz/model/](#))

```
java ✖ Collapse ≡ Unwrap 📄 Copy  
  
package com.example.onlinequiz.model; import jakarta.persistence.*;  
@Entity @Table(name="houses") public class House {  
    @Id @GeneratedValue private Long id; private String name, color;  
    public House() {} public House(String name, String color) { this.name=name;  
this.color=color; }  
    public Long getId() { return id; } public void setId(Long id) { this.id = id; }  
    public String getName() { return name; } public void setName(String name) { this.name  
= name; }  
    public String getColor() { return color; } public void setColor(String color) {  
this.color = color; }  
}
```

FILE 5: Quiz.java ([backend/src/main/java/com/example/onlinequiz/model/](#))

```
java ✖ Collapse ≡ Unwrap 📄 Copy  
  
package com.example.onlinequiz.model; import jakarta.persistence.*; import  
java.util.List; import java.time.LocalDateTime;  
@Entity @Table(name="quizzes") public class Quiz {  
    @Id @GeneratedValue private Long id; private String title, description; private  
    LocalDateTime createdAt = LocalDateTime.now();  
    @OneToMany(mappedBy="quiz") private List<Question> questions; @ManyToOne  
    @JoinColumn(name="house_id") private House house;  
    public Quiz() {} public Quiz(String title, String description, House house) {  
        this.title=title; this.description=description; this.house=house; }  
    // ALL GETTERS/SETTERS (from previous)  
    public House getHouse() { return house; } public void setHouse(House house) {  
        this.house = house; }  
}
```

FILE 6: Question.java ([backend/src/main/java/com/example/onlinequiz/model/](#))

```
java ✖ Collapse ≡ Unwrap 📄 Copy  
  
package com.example.onlinequiz.model; import jakarta.persistence.*; import  
java.util.List;  
@Entity @Table(name="questions") public class Question {  
    @Id @GeneratedValue private Long id; private String text, correctAnswer;  
    @ElementCollection @CollectionTable(name="question_options",  
    joinColumns=@JoinColumn(name="question_id")) @Column(name="option") private List<String>  
    options;  
    @ManyToOne @JoinColumn(name="quiz_id") private Quiz quiz;  
    public Question() {} public Question(String text, List<String> options, String  
    correctAnswer, Quiz quiz) {  
        this.text=text; this.options=options; this.correctAnswer=correctAnswer;  
        this.quiz=quiz;  
    }  
    // ALL GETTERS/SETTERS (from previous)  
}
```

 FILE 7-10: Repositories (`backend/src/main/java/com/example/onlinequiz/repository/`)

java

 Collapse  Unwrap  Copy

```
// QuizRepository.java
package com.example.onlinequiz.repository; import com.example.onlinequiz.model.Quiz;
import org.springframework.data.jpa.repository.JpaRepository; import java.util.List;
public interface QuizRepository extends JpaRepository<Quiz, Long> { List<Quiz>
findByHouseId(Long houseId); }

// QuestionRepository.java
package com.example.onlinequiz.repository; import com.example.onlinequiz.model.Question;
import org.springframework.data.jpa.repository.JpaRepository; import java.util.List;
public interface QuestionRepository extends JpaRepository<Question, Long> {
List<Question> findByQuizId(Long quizId); }

// AdminRepository.java
package com.example.onlinequiz.repository; import com.example.onlinequiz.model.Admin;
import org.springframework.data.jpa.repository.JpaRepository;
public interface AdminRepository extends JpaRepository<Admin, Long> { Admin
findByUsername(String username); }

// HouseRepository.java
package com.example.onlinequiz.repository; import com.example.onlinequiz.model.House;
import org.springframework.data.jpa.repository.JpaRepository;
public interface HouseRepository extends JpaRepository<House, Long> { House
findByName(String name); }
```

FILE 11: ApiController.java (backend/src/main/java/com/example/onlinequiz/controller/)

java

✕ Collapse ≡ Unwrap 📋 Copy

```
package com.example.onlinequiz.controller;
import com.example.onlinequiz.model.*; import com.example.onlinequiz.repository.*; import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*; import java.util.Arrays; import
java.util.List;
@RestController @CrossOrigin(origins="http://localhost:3000") @RequestMapping("/api")
public class ApiController {
    @Autowired private QuizRepository quizRepo; @Autowired private QuestionRepository
questionRepo;
    @Autowired private HouseRepository houseRepo; @Autowired private AdminRepository
adminRepo;

    @PostMapping("/login") public String login(@RequestBody LoginRequest req) {
        Admin admin = adminRepo.findByUsername(req.username); return admin!=null &&
admin.getPassword().equals(req.password) ? "VALID" : "INVALID";
    }
    @GetMapping("/houses") public List<House> getHouses() { return houseRepo.findAll(); }
    @GetMapping("/quizzes") public List<Quiz> getQuizzes() { return quizRepo.findAll(); }
    @GetMapping("/quizzes/house/{houseId}") public List<Quiz>
getQuizzesByHouse(@PathVariable Long houseId) { return quizRepo.findByHouseId(houseId); }
    @PostMapping("/quizzes") public Quiz createQuiz(@RequestBody Quiz quiz) { return
quizRepo.save(quiz); }
    @GetMapping("/quizzes/{id}/questions") public List<Question>
getQuestions(@PathVariable Long id) { return questionRepo.findByQuizId(id); }
    @PostMapping("/quizzes/submit") public int submitQuiz(@RequestBody SubmitRequest req)
{
        int score = 0; for (Answer a : req.answers) { Question q =
questionRepo.findById(a.questionId).get(); if (q.getCorrectAnswer().equals(a.answer))
score++; }
        return score;
    }
}
class LoginRequest { public String username, password; } class SubmitRequest { public
Long quizId; public List<Answer> answers; } class Answer { public Long questionId; public
String answer; }
```

FILE 12: HouseDataLoader.java ([backend/src/main/java/com/example/onlinequiz/](#))

java

✕ Collapse ⇨ Wrap 📄 Copy

```
package com.example.onlinequiz; import com.example.onlinequiz.model.House; import com.examl
import org.springframework.beans.factory.annotation.Autowired; import org.springframework.bc
@Component public class HouseDataLoader implements CommandLineRunner {
    @Autowired private HouseRepository houseRepo; @Override public void run(String... args)
        if (houseRepo.count() == 0) { houseRepo.save(new House("RED", "🔴")); houseRepo.save
    }
}
```

FILE 13: application.properties ([backend/src/main/resources/](#))

properties

✕ Collapse ⇨ Wrap 📄 Copy

```
spring.datasource.url=jdbc:sqlite:./database.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.dialect=org.hibernate.dialect.SQLiteDialect
server.port=8080
```

FILE 14: TestAdminController.java

(backend/src/main/java/com/example/onlinequiz/controller/)

```
java ✕ Collapse ≡ Unwrap 📋 Copy

package com.example.onlinequiz.controller;
import com.example.onlinequiz.model.Admin; import
com.example.onlinequiz.repository.AdminRepository;
import org.springframework.beans.factory.annotation.Autowired; import
org.springframework.stereotype.Controller; import
org.springframework.web.bind.annotation.GetMapping;
@Controller
public class TestAdminController {
    @Autowired private AdminRepository adminRepository;
    @GetMapping("/setup-admin") public String setupAdmin() {
        adminRepository.save(new Admin("admin", "1234")); return "redirect:/api/login";
    }
}
```

FILE 15: LoginRequest.java (backend/src/main/java/com/example/onlinequiz/model/)

```
java ✕ Collapse ≡ Unwrap 📋 Copy

package com.example.onlinequiz.model;
public class LoginRequest { public String username, password; }
```

FILE 16: SubmitRequest.java (backend/src/main/java/com/example/onlinequiz/model/)

```
java ✕ Collapse ⇒ Wrap 📋 Copy

package com.example.onlinequiz.model;
import java.util.List;
public class SubmitRequest { public Long quizId; public List<Answer> answers; }
```

FILE 17: Answer.java (backend/src/main/java/com/example/onlinequiz/model/)

```
java ✕ Collapse ⇒ Wrap 📋 Copy

package com.example.onlinequiz.model;
public class Answer { public Long questionId; public String answer; }
```

FILE 18: index.html (backend/src/main/resources/static/)

```
html Preview Collapse Unwrap Copy

<!DOCTYPE html><html><head><title>House of Hearts</title><script
src="https://unpkg.com/react@18/umd/react.development.js"></script><script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script><script
src="https://unpkg.com/@babel/standalone/babel.min.js"></script></head><body><div
id="root"></div><script type="text/babel" src="/static/App.js"></script></body></html>
```

FILE 19: QuizRepository.java (backend/src/main/java/com/example/onlinequiz/repository/) FULL VERSION

```
java Collapse Wrap Copy

package com.example.onlinequiz.repository;
import com.example.onlinequiz.model.Quiz;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
public interface QuizRepository extends JpaRepository<Quiz, Long> {
    List<Quiz> findByHouseId(Long houseId);
    List<Quiz> findByTitleContainingIgnoreCase(String title);
}
```

FILE 20: QuestionRepository.java (backend/src/main/java/com/example/onlinequiz/repository/) FULL VERSION

```
java Collapse Wrap Copy

package com.example.onlinequiz.repository;
import com.example.onlinequiz.model.Question;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
public interface QuestionRepository extends JpaRepository<Question, Long> {
    List<Question> findByQuizId(Long quizId);
}
```


FRONTEND: React SPA (5 Files)

FILE 21: package.json ([frontend/](#))

json

✕ Collapse ≡ Unwrap 📋 Copy

```
{
  "name": "house-of-hearts-frontend",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "axios": "^1.6.0"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build"
  },
  "proxy": "http://localhost:8080",
  "browserslist": { "production": [">>0.2%", "not dead", "not op_mini all"],
  "development": ["last 1 chrome version", "last 1 firefox version", "last 1 safari version"] }
}
```

FILE 22: index.html (frontend/public/)

html

Preview

Collapse

Unwrap

Copy

```
<!DOCTYPE html><html><head><title>House of Hearts Quiz</title></head><body><div
id="root"></div></body></html>
```

FILE 23: App.js (frontend/src/)

jsx

Collapse

Unwrap

Copy

```
import React, { useState } from 'react'; import axios from 'axios'; import AdminPanel
from './AdminPanel'; import StudentPanel from './StudentPanel'; import './App.css';
function App() { const [isAdmin, setIsAdmin] = useState(false); const [view, setView] =
useState('home'); return (
  <div className="App"><h1>🏠 HOUSE OF HEARTS QUIZ</h1>
    {!isAdmin ? <Login setIsAdmin={setIsAdmin} setView={setView} /> : view ===
    'admin' ? <AdminPanel setView={setView} /> : <StudentPanel setView={setView} />}
  </div>); }
const Login = ({ setIsAdmin, setView }) => { const [username, setUsername] =
useState(''); const [password, setPassword] = useState('');
  const handleLogin = async () => { try { const res = await axios.post('/api/login', {
  username, password }); if (res.data === 'VALID') setIsAdmin(true); } catch { alert('Login
  Failed'); } };
  return (<div><h2>🏠 Admin Login</h2><input value={username} onChange=
  {e=>setUsername(e.target.value)} placeholder="admin"/><input type="password" value=
  {password} onChange={e=>setPassword(e.target.value)} placeholder="1234"/><button onClick=
  {handleLogin}>Login</button></div>); };
export default App;
```

FILE 24: AdminPanel.js (frontend/src/)

jsx

✖ Collapse

≡ Unwrap

📄 Copy

```
import React, { useState, useEffect } from 'react'; import axios from 'axios';
const AdminPanel = ({ setView }) => { const [houses, setHouses] = useState([]); const
[quizzes, setQuizzes] = useState([]); const [title, setTitle] = useState(''); const
[description, setDescription] = useState(''); const [houseId, setHouseId] = useState('');
  useEffect(() => { axios.get('/api/houses').then(res => setHouses(res.data));
  axios.get('/api/quizzes').then(res => setQuizzes(res.data)); }, []);
  const createQuiz = () => { axios.post('/api/quizzes', { title, description, house: {
id: houseId } }).then(() => { setTitle(''); setDescription('');
  axios.get('/api/quizzes').then(res => setQuizzes(res.data)); }); };
  return (<div><h2>🔥 ADMIN PANEL</h2><button onClick=
{()=>setView('home')}>Logout</button>
  <h3>+ Create Quiz</h3><input value={title} onChange={e=>setTitle(e.target.value)}
placeholder="Title"/><input value={description} onChange=
{e=>setDescription(e.target.value)} placeholder="Description"/>
  <select value={houseId} onChange={e=>setHouseId(e.target.value)}><option
value="">House</option>{houses.map(h=><option key={h.id} value={h.id}>{h.name} {h.color}
</option>)}</select>
  <button onClick={createQuiz}>Create</button>
  <h3>📋 Quizzes</h3><ul>{quizzes.map(q => <li key={q.id}>{q.title} - {q.house.name}
{q.house.color}</li>)}</ul></div>); };
export default AdminPanel;
```

FILE 25: StudentPanel.js (`frontend/src/`)

```
jsx
import React, { useState, useEffect } from 'react'; import axios from 'axios';
const StudentPanel = ({ setView }) => { const [houseName, setHouseName] =
useState('RED'); const [quizzes, setQuizzes] = useState([]); const [selectedQuiz,
setSelectedQuiz] = useState(null); const [questions, setQuestions] = useState([]); const
[answers, setAnswers] = useState({});
  useEffect(() => { axios.get(`/api/quizzes/house/${houseName} === 'RED' ? 1 : houseName
=== 'BLUE' ? 2 : 3`).then(res => setQuizzes(res.data)); }, [houseName]);
  const takeQuiz = (quizId) => { setSelectedQuiz(quizId);
  axios.get(`/api/quizzes/${quizId}/questions`).then(res => setQuestions(res.data)); };
  const submitQuiz = () => { axios.post('/api/quizzes/submit', { quizId: selectedQuiz,
answers: Object.entries(answers).map(([qId, ans]) => ({ questionId: +qId, answer: ans })))
  }).then(res => alert(`Score: ${res.data}/10`)); setSelectedQuiz(null); };
  return (<div><h2>STUDENT PORTAL</h2><button onClick=
{()=>setView('admin')}>Admin</button>
    <select value={houseName} onChange={e=>setHouseName(e.target.value)}>
<option>RED</option><option>BLUE</option><option>GREEN</option></select>
    <h3>Quizzes</h3><ul>{quizzes.map(q => <li key={q.id}><button onClick=
{()=>takeQuiz(q.id)}>{q.title}</button></li>)}</ul>
    {selectedQuiz && <div><h3>Taking Quiz</h3>{questions.map(q => <div key={q.id}><p>
{q.text}</p>{q.options.map(opt => <label><input type="radio" name={q.id} value={opt}
onChange={e=>setAnswers({...answers, [q.id]: e.target.value})}>/>{opt}</label>)}</div>)}
<button onClick={submitQuiz}>Submit</button></div>}</div>); };
export default StudentPanel;
```

FILE 26: App.css (`frontend/src/`) (Bonus)

```
css
.App { text-align: center; padding: 20px; } input, select, button { margin: 5px; padding:
8px; } ul { list-style: none; } li { margin: 10px; }
```

BACKEND: 20 FILES (Spring Boot + SQLite Database)

#	FILE	LOCATION	WHAT IT DOES	HOUSE TOURNAMENT ROLE	
1	<code>pom.xml</code>	<code>backend/</code>	BUILD RECIPE	Downloads Spring Boot + Database tools	
2	<code>OnlineQuizApplication.java</code>	<code>java/onlinequiz/</code>	START BUTTON	 RUNS EVERYTHING - 1 click = system live!	
3	<code>Admin.java</code>	<code>model/</code>	ADMIN BLUEPRINT	Stores <code>admin/1234</code> in database	
4	<code>House.java</code>	<code>model/</code>	HOUSE BLUEPRINT	Creates  RED  BLUE  GREEN in database	
5	<code>Quiz.java</code>	<code>model/</code>	QUIZ BLUEPRINT	Stores "Math Champions" + which house owns it	
6	<code>Question.java</code>	<code>model/</code>	QUESTION BLUEPRINT	Stores "2+2=?" + ["A:4","B:5"] + correct "A"	
7	<code>AdminRepository.java</code>	<code>repository/</code>	ADMIN SEARCH	Finds <code>admin/1234</code> when login clicked	
8	<code>HouseRepository.java</code>	<code>repository/</code>	HOUSE SEARCH	Finds all 3 houses for dropdown	
9	<code>QuizRepository.java</code>	<code>repository/</code>	QUIZ SEARCH	Lists RED house quizzes for students	
10	<code>QuestionRepository.java</code>	<code>repository/</code>	QUESTION SEARCH	Loads 10 questions when "TAKE QUIZ"	
11	<code>ApiController.java</code>	<code>controller/</code>	API HIGHWAY	ALL ROUTES: <code>/api/login</code> , <code>/api/quizzes</code> , <code>/api/submit</code>	

12	HouseDataLoader.java	onlinequiz/	HOUSE FACTORY	AUTO-CREATES 3 houses on first run
13	application.properties	resources/	DATABASE SETUP	Creates database.db file + SQLite config
14	TestAdminController.java	controller/	ADMIN CREATOR	/setup-admin = CLICK ONCE → admin ready!
15	LoginRequest.java	model/	LOGIN DATA	Carries username: "admin", password: "1234"
16	SubmitRequest.java	model/	SCORE DATA	Carries student answers to calculate score
17	Answer.java	model/	SINGLE ANSWER	{questionId: 1, answer: "A"} for scoring
18	index.html	static/	REACT LAUNCHER	Loads React UI when you visit localhost:8080
19	QuizRepository FULL	repository/	ADVANCED SEARCH	Filters quizzes by house + title search
20	QuestionRepository FULL	repository/	QUIZ LOADER	Gets all questions for selected quiz

🍷 FRONTEND: 5 FILES (React Single Page App)

#	FILE	LOCATION	WHAT IT DOES	HOUSE TOURNAMENT ROLE	🔗
21	package.json	frontend/	REACT RECIPE	Downloads React + Axios (API caller)	
22	index.html	frontend/public/	REACT HOME	Blank page where React "paints" UI	
23	App.js	src/	MAIN SWITCH	HOME SCREEN: Login → Admin OR Student	
24	AdminPanel.js	src/	ADMIN DASHBOARD	+ Create Quiz → RED House → "Math Champions"	
25	StudentPanel.js	src/	STUDENT PORTAL	RED House → "Take Quiz" → Answer → "Score: 8/10"	

🔥 FLOW DIAGRAM: HOW 25 FILES WORK TOGETHER!

text ✖ Collapse ≡ Wrap 📋 Copy

```
🏆 ADMIN LOGIN (File 23 → 11 → 7 → 3)
↓
+ CREATE QUIZ (24 → 11 → 9 → 5)
↓
🔴 RED HOUSE (12 → 4 → 8)
↓
🍷 STUDENT TAKES (25 → 11 → 10 → 6)
↓
✅ SCORE 8/10 (25 → 11 → 16+17)
↓
🏆 RED HOUSE +10 POINTS!
```